

Работа с библиотечными функциями

Повторение: cdecl

- Как аргументы передаются в функцию в cdecl?
- Как передаются структуры?
- Как возвращаются значения?
- Как возвращаются структуры?

Повторение: вариадические функции (внезапно)

- Что такое вариадические аргументы?
- В каких стандартных функциях они встречаются?
- Как мы работаем с вариадическими аргументами в C?

Работа с libc

- libc – это стандартная библиотека языка C. На компьютере обычно представлена разделяемым объектом, вроде libc6.so.
- Она содержит реализации printf, scanf, malloc и др. стандартных функций
- Чтоб получить корректный исполняемый файл, нужно слинковаться с libc. Компиляторы языка C делают это по умолчанию (еще бы не линковать код на языке с его собственной stdlib). Но мы пишем на ассемблере



Работа с libc

- Чтоб получить корректный исполняемый файл, нужно слинковаться с libc. Компиляторы языка C делают это по умолчанию (еще бы не линковать код на языке с его собственной stdlib). Но мы пишем на ассемблере
- К счастью, это не так сложно.
- Более того, среда SASM и скрипт build.sh уже делают это, поскольку io.inc библиотека – это просто обертка над printf / scanf – стандартными функциями.

Работа с libc

Работа с функциями libc почти ничем не отличается от работы с другими любыми функциями.

Единственное требование: перед вызовом libc-функции, стек должен быть выровнен по 16 байт.

Это необходимо для корректной работы векторных SSE-инструкций, которые активно используются в стандартной библиотеке

Работа с памятью

Работа с памятью происходит с помощью malloc, calloc, free.

1. Мы должны прописать extern используемых функций. Эта директива выполняет ту же роль что и `#include` в C.
2. Не забываем про выравнивание стека. Перед входом в main стек и так выровнен, остается лишь поддерживать его в таком состоянии

```
25 ; We have to EXTERN our functions
24 ; It is much similar to what `#include` does in C
23 extern malloc
22 extern free
21 extern memcpy
20 extern puts
```

Функции с вариадическими аргументами

Поскольку все аргументы передаются через стек, нам все равно сколько аргументов будет на самом деле

Сишные макросы `va_start`, `va_arg`, `va_end` – просто обертки для работы со стеком, т.к. из C мы не можем напрямую работать со стеком.

Давайте посмотрим, во что скомпилируется обычная функция вариадической суммы

<https://godbolt.org/z/csvq9fsaz>

Работа с printf, scanf

Работа с этими функциями ничем не отличается от других, за исключением вариативного числа параметров. Рассмотрим пример

```
; printf("Enter an integer: ");  
push    fmt_hello  
call    printf  
add     esp, 4  
  
; scanf("%d", &x);  
push    x           ; x is ADDRESS HERE  
push    fmt_in  
call    scanf  
add     esp, 8
```

Выравнивание

До сих пор нам не нужно было выравнивать стек. Рассмотрим пример, когда это не так.

```
15 main:
14     push ebp
13     mov  ebp, esp
12
11     push eax      ; alignment is broken now!
10
9      and esp, -16  ; alignment is back!
8
7      push 123
6      push fmt
5      call printf
4      add  esp, 8
3
2      xor  eax, eax
1      leave
23     ret
```